

Distributed and Parallel Computing

MPI Report — 2D Grid & Hybrid OpenMP

Pedro Rio
MSc Data Science and Engineering
Instituto Superior Técnico
97241

Sebastião Maia Serqueira
MSc Data Science and Engineering
Instituto Superior Técnico
97108

Eduardo Machado
MSc Data Science and Engineering
Instituto Superior Técnico
97133

1 Program Overview

The problem is a matrix-factorisation recommender: a sparse user-item matrix A is approximated by $A \approx L \cdot R$, with L of size $users \times features$ and R of size $features \times items$. The factors are fitted by gradient descent over the observed entries, and the final dense product $B = L \cdot R$ yields one recommendation per user. The four stages are *ReadInput*, *InitialLR*, *UpdateLR* (repeated for the requested number of iterations), and *FilterFinalMatrix*.

2 Why a 2D Grid

The simplest MPI design is *replicate-all*: every process keeps full copies of L and R , splits the non-zero list into blocks, computes a partial gradient, and sums it with a global `MPI_Allreduce`. It is correct and short, but it does not scale: the reduced gradient has $O(features \cdot (users + items))$ entries and that volume *does not shrink* as processes are added, while compute per process falls as $1/P$. Communication therefore dominates at scale, and on *wide* instances (a large R) it *anti-scales* — adding processes makes it slower. Memory does not scale either: a single node must hold the whole problem.

Both limits have the same cause — replication — so we distribute the factors over a 2D process grid.

3 The 2D-Grid Version

3.1 Process mesh

`MPI_Dims_create` factors P into a balanced $P_r \times P_c$ grid; `MPI_Cart_create` builds the mesh (with reorder disabled, so a rank's grid id equals its world rank) and `MPI_Cart_sub` derives a *row* and a *column* subcommunicator for each rank.

3.2 Foster methodology

Partitioning. L is split by *user-rows* across the grid rows and R by *item-columns* across the grid columns. A non-zero (u, i) is owned by rank $(userBlock(u), itemBlock(i))$, so each rank holds exactly the L -rows and R -columns its non-zeros touch — memory per rank is $O((users \cdot features + features \cdot items)/\sqrt{P})$.

Communication. Each rank computes a partial gradient from its cell's non-zeros (all data local), then dL is summed across the *row* communicator and dR down the *column* communicator. These are reductions of $\sqrt{P}$ -smaller slices over $\sqrt{P}$ -sized subcommunicators, not a global reduce of the whole gradient.

Agglomeration and Mapping. The non-zeros are agglomerated into one contiguous block per grid cell and routed there once with a bucketed `MPI_Scatterv`; one cell maps to one process.

3.3 Deterministic, communication-free init

`initialLR` is seeded with a fixed value, but a partitioned factor cannot simply seek into the random stream (an R -column slice is strided). The trick is *generate-and-discard*: every rank runs the same `srandom(1)` sequence in the serial fill order but stores only the values that fall in its slice. The slices hold exactly the serial values, with no broadcast of the factors.

3.4 Distributed final stage

$B = L \cdot R$, the mask of known entries, and the per-user arg-max are all computed on the cells: each rank forms its B -block, takes a local arg-max over its item columns, and the results are combined across the row with `MPI_MAXLOC` (smallest item index on ties, matching the serial scan) and gathered to the root in user order. There is no serial-root tail.

4 Hybrid OpenMP Layer

Each rank can fan its local work across OpenMP threads. A *single* parallel region is hoisted around the whole iteration loop (the team forks once), `updateLR` uses orphaned worksharing, and the per-thread gradient conflict is resolved with an array reduction — `reduction(+ : dL[:lN], dR[:rN])` — exactly as in the shared-memory version. MPI stays on the main thread under `MPI_THREAD_FUNNELED`: the two `Allreduce` calls run inside an `omp master` block, fenced by barriers. The default is one thread per rank (pure MPI); threads are explicit opt-in through `OMP_NUM_THREADS`, and one thread reproduces the pure-MPI behaviour exactly.

5 Evaluation

Table ?? compares wall-clock time (seconds, single shared-memory node) of the 2D-grid version against the naive replicate-all design.

The grid version is faster at essentially every process count, and the gap widens with P : replicate-all *regresses* past 4 processes (the dense `Allreduce` ceiling — 10.64s on the wide instance at 8 processes), while the grid version keeps scaling. The 2D reductions are why.

For the hybrid layer on a single node, the picture is more nuanced: pure MPI usually wins, because shared-memory MPI is nearly free while threads add reduction and barrier overhead. Threads pay off only

Table 1: Replicate-all vs. 2D-grid (time, s)

instance	design	1	2	4	8
inst200 (wide)	replicate	7.62	5.26	5.36	10.64
	grid	6.42	3.73	2.22	4.25
inst500	replicate	8.50	4.84	3.28	8.41
	grid	8.32	4.40	2.62	2.64
inst1000	replicate	4.15	2.70	1.89	3.03
	grid	4.16	2.32	1.26	1.42

where adding ranks would over-communicate — on the wide instance at 8 cores, 4 ranks $\times$ 2 threads (2.86s) beats 8 ranks $\times$ 1 thread (5.11s). The hybrid is fundamentally a *cluster* optimisation: it keeps intra-node work off the network.

6 Implications for Parallel and Distributed Computing

6.1 Distribute the data, not just the work

Replicate-all distributes the *work* but not the *data*, so it hits both a memory wall and a communication ceiling. The 2D grid distributes the data: memory and communication per rank both fall as $1/\sqrt{P}$. The general lesson is that a scalable design must shrink each process’s footprint, not only its share of the compute.

6.2 Match the collective to the decomposition

The win is not a faster `Allreduce`; it is a *smaller* one, over a *smaller* communicator. Because a user’s updates are spread only across its grid row and an item’s only down its grid column, the global reduction decomposes into independent row and column reductions. Choosing the communicator to match the data layout is what turns flat communication into communication that scales.

6.3 Remove the serial tail

Computing B and the arg-max on the root would be an $O(\text{users} \cdot \text{items} \cdot \text{features})$ sequential tail that, by Amdahl’s law, caps the speedup. Distributing it over the same grid keeps the whole pipeline parallel.

6.4 The hybrid is for the memory hierarchy

One rank per node (or socket) $\times$ threads-within keeps intra-node work in shared memory and shrinks the process grid, so the inter-node `Allreduces` are cheaper. On a single node, where MPI already uses shared-memory transport, the hybrid is a wash; its value appears across the network, which is exactly where the cost model says fewer, fatter ranks help.

6.5 The thread count cannot be guessed at runtime

The best `OMP_NUM_THREADS` depends on memory bandwidth, NUMA, the interconnect, and the problem shape — machine-specific quantities a single run cannot know. (A formula $T \approx \sqrt{nnz/(uLocal + iLocal)}$ was measured 20–53% off the best in half the cases.) The program therefore defaults to one thread and treats threads as explicit configuration: set the rank/thread split in the job script and confirm it with a short scaling sweep.

6.6 Reproducibility under reordering

The partitioned reductions sum the gradient in a different order than the serial code, so floating-point results differ in the last bits — but the arg-max is robust to that, so the recommendations match the serial output bit-for-bit at every process count, grid shape, and thread count. A deterministic serial reference is what makes this verifiable.